The Happy Path

Immersion Week SF · Day 2 · Session 2.2

55

Minutes

One input reaches one output

What This Session Covers

01 Pick one known-good input

02 Build each step against it

03 Connect the steps end to end

04 Commit the moment it works

What is your one input?

From 2.1 to Now

A stack

What each step runs on

vs

A path

One input, all the way

How We Get There

01

Input

Choose one, freeze it

02

Build

One step at a time

03

Chain

Remove every hand

04

Commit

The second it works

The Next 55 Minutes

10:00

Freeze it

One file, committed, shared

10:10

Build steps

One box at a time, verified

10:35

Connect

Run it from a cold start

10:50

Commit

Tag the version that works

"Make it work. Only then, anything else."

- The Day 2 rule

01

One Known-Good Input

Happy path

The one route where nothing goes wrong

What It Is Not

Not the average case

Not most of your inputs

Not robust to anything

Not what you ship later

Choosing the Input

01 Take a real one

From the actual problem, not invented

02 Check it is boring

Nothing weird, nothing missing

03 Freeze it in the repo

Same bytes for everyone, all day

What Makes One Good

Small enough to read fully

Real enough to be convincing

Boring enough to always work

Fixed, so results compare

Freeze it where everyone can see

```
$ mkdir -p fixtures
$ cp ~/Downloads/report.pdf fixtures/
$ git add fixtures/report.pdf
$ git commit -m "freeze demo input"
```

Today and Not Today

Today

One input, all the way

The steps from your map

Whatever is ugliest

Not today

Edge cases

Error handling

A second input

The Ban List for This Hour

No try or catch anywhere

No configuration files

No second input to test

No renaming or refactoring

Claude Code

An agent that edits and runs your repo

Driving It, Per Step

01 Name the one step

From your map, by its box

02 Give it the contract

What comes in, what goes out

03 Point at the input

The frozen file, by path

Give It What It Needs

The map, as a file in the repo

The contracts between steps

The frozen input, by path

The command that proves it ran

Two Ways to Ask

Vague

"Parse the document"

"Make it handle PDFs"

"Clean up the output"

Specific

"Read fixtures/report.pdf"

"Return title and date"

"Write it to out.json"

A request worth making

Read fixtures/report.pdf.
Pull out title, author, date.
Write them to out/step1.json.
Run it and show me the file.

Three Rules for Driving

One step per ask

Never two boxes of the map at once

Make it run

Ask for the output, not a description

Reset, do not nudge

If two tries fail, restate the whole ask

When It Goes Sideways

It invented a file — name paths

It over-built — say what to skip

It cannot run — give it the command

It loops — restate from scratch

Check Each Step Before Moving On

You ran it yourself, just now

The output file exists

You read the output

It matches the contract

02

Connect the Steps

Where do hands still touch?

Manual step

Anything a person does between two steps

Manual Steps That Hide

Copying output into the next box

Renaming a file by hand

Pasting into a chat window

Knowing to run them in order

Two Kinds of Demo

Narrated

And now I open this

vs

Run once

One command, done

Connecting, in Order

01 Write the contracts down

What each step leaves behind

02 Chain two, then three

Add one box at a time

03 Run it from cold

Delete the outputs and start over

Write Down the Seams

Step	Leaves behind
Ingest	out/raw.json
Extract	out/fields.json
Reason	out/answer.md
Render	out/index.html

The whole path, one command

```
$ rm -rf out/ && mkdir out
$ python run.py fixtures/report.pdf
... 4 steps ok
$ open out/index.html
```

The Seam Is Where It Breaks

One step writes, the next cannot read

A field is named two ways

One step assumes a list

A path is right on one machine

Debugging a Seam

01 Open both files

What was written, what was expected

02 Fix the later step

Adapt downstream, not upstream

03 Rerun from cold

Never trust a partial run

Connect Early or Late

	Connect early	Connect late
Surprises	**Now, small**	Late, large
Rework	Cheap	Expensive
Demo risk	**Low**	High

Four Smells

Smell	What it means
Only Ana	One person can run it
Warm only	Fails from a clean start
In order	You must know the sequence
One box	A step still done by hand

If You Are Behind

01 Cut a box, not a corner

Stub a whole step, keep the path whole

02 Shrink the input

One page instead of the whole report

03 Never drop the commit

A short path committed beats a long one lost

Done Means All Four

One command, start to finish

Runs from an empty output dir

Someone else ran it

The output is the real thing

"If hands touch it, it is not connected."

- Objective two, restated

03

Commit It Immediately

Works

It ran end to end, cold, in front of you

Why Right Now

This is the version that works

The next change may break it

You will not remember what changed

A demo needs a known-good commit

The Commit Ritual

01 Stop typing

Hands off the keyboard, it works

02 Commit and push

Message says what now works

03 Then keep going

Anything new lands on top of it

Sixty seconds of insurance

```
$ git add run.py steps/ fixtures/
$ git commit -m "happy path runs cold"
$ git push -u origin main
$ git tag happy-path && git push --tags
```

What the repo looks like now

```
happy-path/
├── fixtures/
│   └── report.pdf
├── steps/
│   ├── extract.py
│   └── render.py
├── run.py
└── README.md
```

What the Message Should Say

What input it accepts

What output it produces

What is still stubbed

Not how clever it is

Before and After the Commit

Before committing	After committing
Nothing else at all	Everything you want
No tidying, no renaming	Break it freely
No one more small fix	You can always get back

What the Commit Buys You

A place to stand

Every later change can be undone to here

A shared build

Teammates start from what already works

A safe demo

A known-good tag if tomorrow goes wrong

Then, and Only Then

Add a second input

Handle the obvious failure

Make the output look right

Start on the demo wrapper

Write This Down Now

Question	Your answer
The frozen input	…
The one command	…
The known-good tag	…
Still done by hand	…

Leaving This Session

One input, frozen in the repo

One command runs it all

It works from a cold start

It is committed and pushed

Could a stranger run it?

Next: Making It Demoable

A front end a stranger can drive

A public URL, not your laptop

A three-minute demo script

An honest list of what is faked